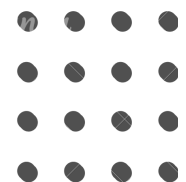




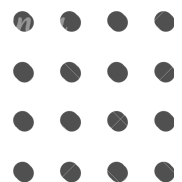
VMES Grinds





Pilot Program - Week 4 - Day 2

Heap





Leetcode: 621 Task Scheduler



Moe Myint Mo San



621. Task Scheduler

Solved

Medium

Topics

Companies

Hint

You are given an array of CPU `tasks`, each labeled with a letter from A to Z, and a number `n`. Each CPU interval can be idle or allow the completion of one task. Tasks can be completed in any order, but there's a constraint: there has to be a gap of **at least** `n` intervals between two tasks with the same label.

Return the **minimum** number of CPU intervals required to complete all tasks.

Example 1:

Input: `tasks = ["A","A","A","B","B","B"], n = 2`

Output: 8

Explanation: A possible sequence is: A -> B -> idle -> A -> B -> idle -> A -> B.

After completing task A, you must wait two intervals before doing A again. The same applies to task B. In the 3rd interval, neither A nor B can be done, so you idle. By the 4th interval, you can do A again as 2 intervals have passed.



Example 2:

Input: tasks = ["A", "C", "A", "B", "D", "B"], n = 1

Output: 6

Explanation: A possible sequence is: A -> B -> C -> D -> A -> B.

With a cooling interval of 1, you can repeat a task after just one other task.

Example 3:

Input: tasks = ["A", "A", "A", "B", "B", "B"], n = 3

Output: 10

Explanation: A possible sequence is: A -> B -> idle -> idle -> A -> B -> idle -> idle -> A -> B.

There are only two types of tasks, A and B, which need to be separated by 3 intervals. This leads to idling twice between repetitions of these tasks.



Constraints:

- $1 \leq \text{tasks.length} \leq 10^4$
- `tasks[i]` is an uppercase English letter.
- $0 \leq n \leq 100$



Problem Explanation Thought Process





Initial Thought Process

To schedule tasks efficiently:

- The challenge comes from tasks with high frequency.
- If too frequent, they force idle gaps.
- If enough other tasks exist, they fill the gaps → no idle time.
- Two main strategies:
 - a. Simulate scheduling with greedy selection.
 - b. Calculate using frequency-based formula.



Algorithm Evolution Overview



Approach	Time	Space	Notes
Simulation (Heap + Cooldown)	$O(I \log K)$	$O(k)$	Visual, step-by-step scheduling
Formula (Counting)	$O(T)$	$O(1)$	Fastest, interview-friendly



Simulation (Heap + Cooldown)





Idea:

- Always pick the most frequent available task.
- After executing, put it into a cooldown list until allowed again.
- Repeat until heap + cooldown empty.

Why It Works

- Greedy choice: schedule the most frequent first to avoid future idle gaps.
- Cooldown enforces spacing: tasks only re-enter heap once valid.
- Simulation mimics real CPU behavior → produces valid minimal schedule.



```
import heapq

def leastInterval(tasks, n):
    """
    :type tasks: List[str]
    :type n: int
    :rtype: int
    """
    if n == 0:
        return len(tasks)
    tasks.sort()
    fqs = []

    for i in range(0, len(tasks)):
        if i == 0:
            fqs = [0]
        if i > 0 and tasks[i] != tasks[i-1]:
            fqs.append(0)
        fqs[-1] += 1
    fqs = [-f for f in fqs]
    heapq.heapify(fqs)

    ans = 0
    time = []
    cache = []
```




```
        fqs[-1] += 1
    fqs = [-f for f in fqs]
    heapq.heapify(fqs)

    ans = 0
    time = []
    cache = []

    while len(fqs) > 0 or len(cache) > 0:
        ans += 1

        if len(time) > 0 and time[0] == ans:
            heapq.heappush(fqs, -cache[0])
            cache = cache[1:]
            time = time[1:]

        if len(fqs) == 0:
            continue

        temp = -heapq.heappop(fqs)
        if temp > 1:
            time.append(ans + n + 1)
            cache.append(temp - 1)

    return ans
```



Time Complexity (Brute Force)

- Time: $O(I \log k)$
- I = total intervals (tasks + idle)
- $k \leq 26$ (unique letters)
- Space: $O(k)$ for heap + cooldown list.



Formula Method





Idea:

- Let maxf = highest task frequency.
- Let k = number of tasks that appear maxf times.
- Build blocks of size $(n + 1)$ for $\text{maxf} - 1$ cycles.

Formula:

$\text{layout} = (\text{maxf} - 1) * (n + 1) + k$
 $\text{answer} = \max(\text{len}(\text{tasks}), \text{layout})$



```
class Solution(object):
    def leastInterval(self, tasks, n):
        if n == 0:
            return len(tasks)

        cache = {}
        for t in tasks:
            cache[t] = cache.get(t, 0) + 1

        maxf = max(cache.values())
        k = sum(1 for v in cache.values() if v == maxf)

        return max(len(tasks), (maxf - 1) * (n + 1) +
k)
```



Why This Works

- The most frequent tasks force mandatory spacing.
- $\text{maxf} - 1$ groups must each have n gaps.
- Final block contains k tasks tied for highest frequency.
- Layout gives minimum required size.
- If other tasks exist, they fill gaps, so result becomes $\max(\text{len}, \text{layout})$.

Time Complexity (Brute Force)

- Time: $O(T)$ to count frequencies.
- Space: $O(1)$ (fixed 26-char alphabet).
- Very small, constant overhead $\rightarrow$ best for interviews.



Extra Study Material –Visualizations–





Task Scheduler - Math Formula Approach Summary

The Formula

$$\text{Answer} = \max(\text{len}(\text{tasks}), (\text{maxf} - 1) * (\text{n} + 1) + \text{k})$$

Part	Meaning
<code>maxf</code>	Highest frequency among all tasks
<code>k</code>	Count of tasks that have the max frequency
<code>(maxf - 1)</code>	Number of "full frames" (gaps between max-frequency tasks)
<code>(n + 1)</code>	Size of each frame (1 task + n cooldown slots)
<code>max(len(tasks), ...)</code>	Ensures answer is never less than total tasks



For `tasks = ["A","A","A","B","B","B"]`, `n = 2`:

A _ _ A _ _ A $\leftarrow$ 3 A's need 2 gaps between them

- **Frames:** `(maxf - 1)` = 2 frames (between the A's)
- **Frame size:** `(n + 1)` = 3 slots each (A + 2 cooldown slots)
- **Last row:** `k` = 2 tasks (A and B) fill the final position

Frame 1: [A] [B] [_]

Frame 2: [A] [B] [_]

Last: [A] [B]

Total = 2 * 3 + 2 = 8



Input: `tasks = ["A","A","A","B","B","B"]` , `n = 2`

Step 1: Count frequencies

```
cache = {'A': 3, 'B': 3}
maxf = 3
k = 2 (both A and B appear 3 times)
```

Step 2: Build the frame structure

```
Frame 1: [A] [B] [_]      (size = n + 1 = 3)
Frame 2: [A] [B] [_]      (size = n + 1 = 3)
Last:    [A] [B]          (size = k = 2)
```

Step 3: Calculate

```
(maxf - 1) * (n + 1) + k
= (3 - 1) * (2 + 1) + 2
= 2 * 3 + 2
= 8
```



Example Trace

For `tasks = ["A", "A", "A", "B", "B", "B"]` :

Step	<code>t</code>	<code>cache.get(t, 0)</code>	<code>+ 1</code>	<code>cache</code> after
1	"A"	0 (not found)	1	{ 'A': 1 }
2	"A"	1	2	{ 'A': 2 }
3	"A"	2	3	{ 'A': 3 }
4	"B"	0 (not found)	1	{ 'A': 3, 'B': 1 }
5	"B"	1	2	{ 'A': 3, 'B': 2 }
6	"B"	2	3	{ 'A': 3, 'B': 3 }



Why Do We Need `max()` ?

Case 1: Few Task Types (Formula Wins)

```
tasks = ["A","A","A","B","B","B"], n = 2
```

```
Frame 1: [A] [B] [_]    <- idle needed
```

```
Frame 2: [A] [B] [_]    <- idle needed
```

```
Last:    [A] [B]
```

```
Formula = 8
```

```
len(tasks) = 6
```

```
Answer = max(6, 8) = 8
```

Not enough task types to fill gaps, so idles are required.



Case 2: Many Task Types (len(tasks) Wins)

```
tasks = ["A","A","A","B","C","D","E","F","G"], n = 2
```

```
Sequence: A-B-C-A-D-E-A-F-G  
          1 2 3 4 5 6 7 8 9
```

```
Formula = 7
```

```
len(tasks) = 9
```

```
Answer = max(9, 7) = 9
```

Plenty of different tasks fill all cooldown slots, no idles needed.



Complexity Comparison



Approach	Time	Space	Notes
Simulation (Heap + Cooldown)	$O(I \log K)$	$O(k)$	Clear scheduling visualization
Formula (Counting)	$O(T)$	$O(1)$	Fastest, simplest, optimal



Key Takeaway





This is fundamentally a frequency-driven scheduling problem:

- The most frequent task dictates structure.
- Simulation shows the true execution timeline.
- Formula gives the minimal time mathematically.
- Correct reasoning = understand spacing, frequency, and idle filling.



Thank You !

